

Comparação entre Algoritmo Genético e Otimização por Enxame de Partículas para o Problema do Caixeiro Viajante

Artur Henrique Lara, Bryan Luiz Veloso da Silva,
Cassiuscray Felipe dos Santos, Pedro Henrique Faria Moura do Altíssimo

¹Departamento de Ciência da Computação
Universidade Federal de São João del-Rei (UFSJ) – São João del-Rei, MG – Brasil

arturhlara@aluno.ufsj.edu.br, bryanveloso@aluno.ufsj.edu.br,
cassiuscrayfelipe@aluno.ufsj.edu.br, pehfaaria@aluno.ufsj.edu.br

Resumo. *O Problema do Caixeiro Viajante (TSP) é um problema clássico de otimização combinatória, com aplicações em logística, projeto de circuitos e roteamento. Este trabalho compara duas meta-heurísticas populacionais aplicadas ao TSP: Algoritmo Genético (GA) e Otimização por Enxame de Partículas (PSO). Ambos os algoritmos foram implementados utilizando representação por caminho (path), com o GA empregando Order Crossover (OX) e mutação por posição, e o PSO utilizando operadores baseados em swaps, com inércia decrescente e clamping de velocidade. Os experimentos foram conduzidos em quatro instâncias com tamanhos variados (12 a 101 cidades), incluindo instâncias clássicas da biblioteca TSPLIB. Foram realizadas 30 execuções por algoritmo em cada instância, e a comparação foi validada estatisticamente com o teste de Wilcoxon-Mann-Whitney. Os resultados indicam que o GA apresentou desempenho superior em todas as instâncias testadas, com diferença estatisticamente significativa ($p < 0,01$), e que o desempenho relativo do PSO degrada à medida que o tamanho da instância aumenta.*

1. Introdução

O Problema do Caixeiro Viajante (*Travelling Salesman Problem* – TSP) é um dos problemas mais estudados em otimização combinatória [Applegate et al. 2007]. Sua formulação é simples: dado um conjunto de cidades e as distâncias entre cada par delas, deve-se determinar o menor circuito que visita todas as cidades exatamente uma vez e retorna ao ponto de origem. Apesar da simplicidade da descrição, o TSP pertence à classe NP-difícil, o que significa que não se conhece algoritmo de tempo polinomial capaz de resolvê-lo exatamente para instâncias arbitrariamente grandes.

A relevância prática do TSP é considerável: aplicações incluem roteamento de veículos, sequenciamento de operações em circuitos integrados, programação de máquinas industriais e logística [Laporte 1992]. Esses contextos exigem soluções de boa qualidade obtidas em tempo razoável, o que motivou o desenvolvimento de meta-heurísticas, técnicas que buscam soluções aproximadas explorando o espaço de busca de forma inteligente.

Entre as meta-heurísticas mais utilizadas estão as inspiradas em fenômenos naturais. O **Algoritmo Genético** (*Genetic Algorithm* – GA), proposto por [Holland 1975], baseia-se nos princípios da evolução darwiniana, simulando seleção natural, reprodução

e mutação. A **Otimização por Enxame de Partículas** (*Particle Swarm Optimization* – PSO), proposta por [Kennedy and Eberhart 1995], inspira-se no comportamento coletivo de bandos de pássaros e cardumes de peixes, com partículas que se movimentam no espaço de busca influenciadas por suas próprias experiências e pelas experiências do grupo.

Este trabalho tem como objetivo **implementar e comparar** GA e PSO aplicados ao TSP, investigando o desempenho de cada algoritmo em termos de qualidade da solução encontrada, velocidade de convergência e consistência entre execuções. A comparação é realizada de forma estatisticamente rigorosa, com múltiplas execuções e teste de significância. As principais contribuições do trabalho são:

- Implementação completa de GA com OX e mutação por posição, e de PSO baseado em *swaps* com inércia decrescente e *clamping* de velocidade;
- Análise estatística comparativa com 30 execuções por algoritmo, validada pelo teste de Wilcoxon-Mann-Whitney;
- Experimentos em quatro instâncias de tamanhos variados, incluindo instâncias clássicas da TSPLIB com ótimos conhecidos.

O restante do artigo está organizado da seguinte forma. A Seção 2 apresenta o referencial teórico. A Seção 3 detalha a metodologia adotada. A Seção 4 apresenta os resultados experimentais e a discussão. A Seção 5 encerra o trabalho com conclusões e direções futuras.

2. Referencial Teórico

2.1. Problema do Caixeiro Viajante

Formalmente, o TSP pode ser definido sobre um grafo completo $G = (V, E)$, onde $V = \{v_1, v_2, \dots, v_n\}$ é o conjunto de n cidades e E é o conjunto de arestas, com uma distância d_{ij} associada a cada aresta (v_i, v_j) . Uma *rota* é uma permutação dos vértices $\pi = (\pi_1, \pi_2, \dots, \pi_n)$, e seu custo total é dado por:

$$C(\pi) = \sum_{i=1}^{n-1} d_{\pi_i \pi_{i+1}} + d_{\pi_n \pi_1} \quad (1)$$

O termo final $d_{\pi_n \pi_1}$ representa o retorno à cidade de origem. O objetivo é encontrar a permutação π^* que minimiza $C(\pi)$. Neste trabalho, considera-se o TSP simétrico ($d_{ij} = d_{ji}$) com distâncias Euclidianas em duas dimensões:

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \quad (2)$$

O número de rotas distintas possíveis é $(n-1)!/2$, tornando inviável a enumeração para instâncias de tamanho moderado.

2.2. Representação de Soluções

A escolha da representação é fundamental para o desempenho de meta-heurísticas aplicadas ao TSP. [Larrañaga et al. 1999] revisam as principais representações utilizadas em algoritmos evolutivos para o problema, das quais destacam-se:

- **Path:** a rota é representada diretamente como uma sequência de cidades, na ordem de visitação. É a representação mais intuitiva e a mais utilizada na literatura.
- **Adjacency:** a posição i do vetor indica qual cidade é visitada *imediatamente após* a cidade i . Operadores genéticos clássicos frequentemente geram soluções inválidas (com sub-ciclos).
- **Ordinal:** utiliza uma lista de referência e índices que indicam qual posição extrair a cada passo. Tem a vantagem de garantir validade com operadores clássicos, mas perde informação estrutural.
- **Matricial:** matriz binária $n \times n$ que codifica precedências ou adjacências. Permite operadores ricos em informação estrutural, mas tem custo de memória $O(n^2)$ e gera soluções inválidas com frequência.

Neste trabalho optou-se pela representação **path** pelos seguintes motivos: (i) é intuitiva e simples de implementar; (ii) é a representação mais utilizada na literatura tanto para GA quanto para PSO aplicados ao TSP, facilitando comparação com trabalhos anteriores; e (iii) existem operadores genéticos especializados, como o *Order Crossover*, que preservam a validade das rotas e exploram bem o espaço de busca.

2.3. Algoritmo Genético

O GA [Holland 1975, Goldberg 1989] é uma meta-heurística populacional inspirada na evolução biológica. Mantém-se uma população de soluções candidatas (indivíduos) que evolui ao longo de gerações por meio de operadores de seleção, cruzamento e mutação. O Algoritmo 1 apresenta a estrutura geral do GA aplicado ao TSP.

Algorithm 1 Algoritmo Genético para o TSP

```

1: Gerar população inicial  $P$  de rotas aleatórias
2: Avaliar custo de cada indivíduo em  $P$ 
3: while critério de parada não atingido do
4:    $P' \leftarrow \{\text{melhor indivíduo de } P\}$  ▷ Elitismo
5:   while  $|P'| < |P|$  do
6:      $p_1 \leftarrow \text{seleção por torneio em } P$ 
7:      $p_2 \leftarrow \text{seleção por torneio em } P$ 
8:     if  $\text{rand}() < \text{taxa\_crossover}$  then
9:        $f \leftarrow \text{OX}(p_1, p_2)$ 
10:    else
11:       $f \leftarrow p_1$ 
12:    end if
13:     $f \leftarrow \text{mutação por posição em } f$ 
14:     $P' \leftarrow P' \cup \{f\}$ 
15:  end while
16:   $P \leftarrow P'$ 
17: end while
18: return melhor indivíduo encontrado

```

2.3.1. Operadores

Seleção por torneio: k indivíduos são selecionados aleatoriamente da população, e o de menor custo é escolhido como pai. Esse operador equilibra pressão seletiva (favorecendo soluções boas) e diversidade.

Order Crossover (OX): operador de cruzamento específico para representações de permutação. Funciona em três etapas: (i) seleciona-se um segmento aleatório do pai 1, que é copiado para o filho na mesma posição; (ii) as cidades restantes são inseridas no filho na ordem em que aparecem no pai 2, ignorando as já presentes; (iii) a inserção começa após a posição final do segmento copiado, dando a volta circular se necessário. O OX preserva tanto a ordem relativa quanto a posição absoluta de subsequências dos pais.

Mutação por posição: cada posição da rota tem probabilidade $1/n$ (onde n é o número de cidades) de ser trocada com outra posição aleatória. Essa formulação produz, em média, uma troca por mutação, mas permite múltiplas trocas em alguns casos, aumentando a diversidade exploratória.

Elitismo: o melhor indivíduo da geração atual é preservado integralmente na próxima geração, garantindo que a qualidade da melhor solução nunca decresça ao longo da execução.

2.4. Otimização por Enxame de Partículas

O PSO [Kennedy and Eberhart 1995] é uma meta-heurística inspirada no comportamento social de enxames. Cada *partícula* representa uma solução candidata e mantém duas informações principais: sua melhor posição já visitada ($pBest$) e a melhor posição encontrada por qualquer partícula do enxame ($gBest$). A cada iteração, a partícula atualiza sua *velocidade* e sua *posição* segundo:

$$v_i^{t+1} = w \cdot v_i^t + c_1 r_1 (pBest_i - x_i^t) + c_2 r_2 (gBest - x_i^t) \quad (3)$$

$$x_i^{t+1} = x_i^t + v_i^{t+1} \quad (4)$$

onde w é o coeficiente de inércia, c_1 e c_2 são os coeficientes cognitivo e social, e $r_1, r_2 \in [0, 1]$ são números aleatórios.

2.4.1. PSO discreto para o TSP

A formulação clássica do PSO opera em espaços contínuos, sendo necessárias adaptações para o TSP, que é discreto. Neste trabalho adotou-se a abordagem **baseada em swaps** (*swap-based PSO*), na qual:

- **Posição** é uma rota válida (permutação de cidades);
- **Velocidade** é uma sequência ordenada de pares (i, j) representando trocas de posições;
- **Diferença entre posições** ($pBest - x$ ou $gBest - x$) é calculada como a sequência de swaps que transforma uma rota na outra;

- **Multiplicação por escalar** (no produto $c \cdot r \cdot \text{swaps}$) é interpretada probabilisticamente: cada swap é mantido com probabilidade igual ao escalar;
- **Soma de velocidades** é a concatenação das sequências de swaps;
- **Aplicação da velocidade** é a execução sequencial dos swaps sobre a rota.

A escolha pela abordagem baseada em swaps, em detrimento de alternativas como *Random Keys*, foi motivada por sua maior aderência à natureza combinatória do TSP e por ser amplamente utilizada na literatura [Clerc 2004]. O Algoritmo 2 apresenta a estrutura do PSO implementado.

Algorithm 2 PSO baseado em swaps para o TSP

```

1: Inicializar partículas como rotas aleatórias
2: Inicializar velocidades como sequências vazias
3: Avaliar custo inicial; definir  $pBest$  e  $gBest$ 
4: for  $t = 1$  até máximo de iterações do
5:    $w_t \leftarrow w_{max} - (w_{max} - w_{min}) \cdot \frac{t-1}{T-1}$ 
6:   for cada partícula  $i$  do
7:      $v_{cog} \leftarrow \text{swaps de } x_i \text{ para } pBest_i \text{ com prob. } c_1 r_1$ 
8:      $v_{soc} \leftarrow \text{swaps de } x_i \text{ para } gBest \text{ com prob. } c_2 r_2$ 
9:      $v_i \leftarrow w_t \cdot v_i + v_{cog} + v_{soc}$  ▷ Concatenação
10:     $v_i \leftarrow \text{clamp}(v_i, v_{max})$ 
11:     $x_i \leftarrow \text{aplicar swaps } v_i \text{ em } x_i$ 
12:    Atualizar  $pBest_i$  e  $gBest$  se houver melhora
13:   end for
14: end for
15: return  $gBest$ 

```

Inércia decrescente: adotou-se a estratégia de *inércia linearmente decrescente* [Shi and Eberhart 1998], na qual o coeficiente w decai linearmente de w_{max} no início para w_{min} no final do processo. Isso favorece a exploração nas primeiras iterações e a exploração nas finais.

Clamping de velocidade: para evitar que a velocidade cresça descontroladamente (o que faria a partícula essencialmente embaralhar sua rota a cada iteração), aplica-se um limite máximo v_{max} ao tamanho da lista de swaps. Quando excedido, uma amostragem aleatória de v_{max} swaps é mantida, preservando a ordem original.

Topologia: adotou-se a topologia global (*Star*), na qual todas as partículas compartilham o mesmo $gBest$. Essa topologia favorece convergência rápida e é a mais utilizada em implementações canônicas do PSO [Kennedy and Eberhart 1995].

2.5. Trabalhos Relacionados

A aplicação de meta-heurísticas evolutivas ao TSP tem extensa literatura. [Larrañaga et al. 1999] apresentam uma revisão abrangente de representações e operadores para GAs aplicados ao TSP. [Clerc 2004] propõe um dos primeiros modelos de PSO discreto para problemas combinatórios. [Shi and Eberhart 1998] introduzem a inércia decrescente como mecanismo de balanceamento entre exploração e exploração no PSO. Comparações entre GA e PSO para o TSP foram realizadas em diversos trabalhos,

geralmente apontando vantagens contextuais de cada abordagem dependendo do tamanho da instância e dos parâmetros adotados.

3. Metodologia

3.1. Implementação

Ambos os algoritmos foram implementados em **Python 3**, utilizando apenas bibliotecas padrão da linguagem para a lógica algorítmica. A biblioteca `matplotlib` foi utilizada para visualização, e `scipy` para o teste estatístico. A função de avaliação de rota utiliza uma **matriz de distâncias** pré-computada na inicialização, evitando recálculos redundantes da distância Euclidiana e reduzindo significativamente o tempo de execução em instâncias maiores.

3.2. Hiperparâmetros

A Tabela 1 apresenta os hiperparâmetros adotados em todos os experimentos. Os valores foram definidos com base em referências da literatura [Shi and Eberhart 1998, Goldberg 1989] e em experimentos preliminares.

Tabela 1. Hiperparâmetros utilizados nos experimentos.

Algoritmo	Parâmetro	Valor
GA	Tamanho da população	100
	Taxa de crossover	0,85
	Taxa de mutação (por posição)	$1/n$
	Tamanho do torneio	3
PSO	Número de partículas	100
	Inércia (w_{max}, w_{min})	(0,9; 0,4)
	Coeficiente cognitivo (c_1)	1,5
	Coeficiente social (c_2)	1,5
	v_{max} (fator de n)	0,5
Ambos	Iterações máximas	500
	Critério de parada por estagnação	100 iterações

3.2.1. Justificativa das principais escolhas

Tamanho da população (100): valores entre 50 e 200 são comumente reportados na literatura para o TSP com instâncias de tamanho moderado [Goldberg 1989]. Optou-se por 100 como compromisso entre diversidade da população (que reduz o risco de convergência prematura) e custo computacional. Para ambos os algoritmos foi utilizado o mesmo valor, garantindo equidade na comparação.

Número máximo de iterações (500): dimensionado de modo a permitir convergência completa nas instâncias testadas. Observou-se nos experimentos preliminares que, para todas as instâncias avaliadas, o melhor custo já se estabilizava antes da iteração

400, indicando que 500 iterações são suficientes para esgotar o potencial dos algoritmos sob os hiperparâmetros adotados.

Critério de parada por estagnação (100 iterações): adotou-se um critério adicional ao número máximo de iterações: a execução é interrompida se o melhor custo não apresentar melhora durante 100 iterações consecutivas. Esse valor corresponde a 20% do total de iterações, sendo amplo o suficiente para evitar paradas prematuras (que descartariam melhorias ainda possíveis) e curto o suficiente para reduzir desperdício computacional quando o algoritmo já convergiu. Esse critério foi aplicado igualmente aos dois algoritmos para preservar a comparabilidade.

Coefficientes do PSO ($c_1 = c_2 = 1,5$): valores comumente adotados, com $c_1 = c_2$ representando equilíbrio entre as influências cognitiva (memória pessoal) e social (memória do enxame). A estratégia de *inércia linearmente decrescente* de 0,9 a 0,4 segue a recomendação clássica de [Shi and Eberhart 1998], favorecendo exploração no início e exploração ao final.

Taxa de crossover (0,85) e taxa de mutação ($1/n$): valores recorrentes na literatura para GA aplicado ao TSP. A taxa de mutação por posição igual a $1/n$ produz, em média, uma troca por mutação, equivalente em magnitude à mutação por indivíduo tradicional, mas com maior potencial de exploração ao permitir múltiplas trocas em alguns indivíduos.

3.3. Instâncias

Os experimentos foram conduzidos em quatro instâncias do TSP, descritas na Tabela 2. A instância `tsp_12` foi fornecida na especificação do trabalho. As instâncias `berlin52`, `kroA100` e `eil101` foram obtidas da biblioteca TSPLIB [Reinelt 1991], que disponibiliza instâncias clássicas amplamente utilizadas em pesquisas sobre o TSP e que possuem soluções ótimas conhecidas, permitindo avaliar a qualidade absoluta das soluções encontradas pelos algoritmos.

Tabela 2. Instâncias utilizadas nos experimentos.

Instância	Origem	Cidades	Ótimo conhecido
tsp_12	Especificação do trabalho	12	29,25 (*)
berlin52	TSPLIB	52	7.542
kroA100	TSPLIB	100	21.282
eil101	TSPLIB	101	629

(*) Custo ótimo verificado experimentalmente.

3.4. Análise Estatística

Devido à natureza estocástica dos algoritmos, cada algoritmo foi executado **30 vezes** em cada instância, e foram coletadas as seguintes métricas: melhor custo encontrado, pior custo encontrado, média, desvio padrão e tempo médio de execução.

Para validar a significância das diferenças observadas, aplicou-se o **teste de Wilcoxon-Mann-Whitney** [Mann and Whitney 1947], um teste não-paramétrico apropriado para comparar duas amostras independentes sem assumir normalidade na

distribuição dos custos. Adotou-se nível de significância $\alpha = 0,05$: se o p -valor for inferior a 0,05, rejeita-se a hipótese nula de que os dois algoritmos têm desempenho equivalente, e considera-se a diferença estatisticamente significativa.

4. Experimentos e Resultados

4.1. Resultados Quantitativos

A Tabela 3 apresenta as métricas obtidas pelos dois algoritmos nas quatro instâncias testadas, sobre 30 execuções independentes. A Tabela 4 apresenta os resultados do teste de Wilcoxon-Mann-Whitney comparando os custos finais obtidos pelos dois algoritmos.

Tabela 3. Resultados de 30 execuções por algoritmo em cada instância. Valores em negrito indicam o melhor desempenho.

Instância	Alg.	Melhor	Pior	Média	Desvio	Tempo (s)
tsp_12	GA	29,25	31,83	29,49	0,64	0,62
	PSO	29,25	33,61	29,91	1,13	0,22
berlin52	GA	11.482,26	13.694,72	12.496,26	658,52	2,74
	PSO	11.629,35	14.806,62	13.049,81	774,79	3,30
kroA100	GA	64.085,88	76.860,80	71.724,43	3.426,66	4,60
	PSO	82.264,73	117.702,75	97.062,59	10.505,67	9,23
eil101	GA	1.450,34	1.872,39	1.623,33	86,57	4,46
	PSO	1.818,15	2.406,73	2.092,07	126,75	9,47

Tabela 4. Resultados do teste de Wilcoxon-Mann-Whitney ($\alpha = 0,05$).

Instância	Estatística U	p -valor	Significativo?
tsp_12	281,00	0,003854	Sim
berlin52	267,00	0,006972	Sim
kroA100	0,00	< 0,000001	Sim
eil101	2,00	< 0,000001	Sim

4.2. Distância em Relação ao Ótimo Conhecido

Para as instâncias da TSPLIB, é possível avaliar a qualidade absoluta das soluções obtidas por meio do *gap percentual* em relação ao ótimo conhecido, definido por:

$$\text{gap}(\%) = \frac{C_{\text{obtido}} - C_{\text{otimo}}}{C_{\text{otimo}}} \times 100 \quad (5)$$

A Tabela 5 apresenta o *gap* médio das soluções encontradas pelos dois algoritmos.

4.3. Análise Visual

A Figura 3 apresenta a distribuição dos custos finais obtidos pelos dois algoritmos na instância kroA100, na qual a separação entre as distribuições é evidente, corroborando com a análise do algoritmo e com o resultado do teste estatístico.

Tabela 5. Gap percentual médio em relação ao ótimo conhecido.

Instância	Ótimo	Gap GA (%)	Gap PSO (%)
berlin52	7.542	65,69	73,03
kroA100	21.282	237,02	356,17
eil101	629	158,08	232,60

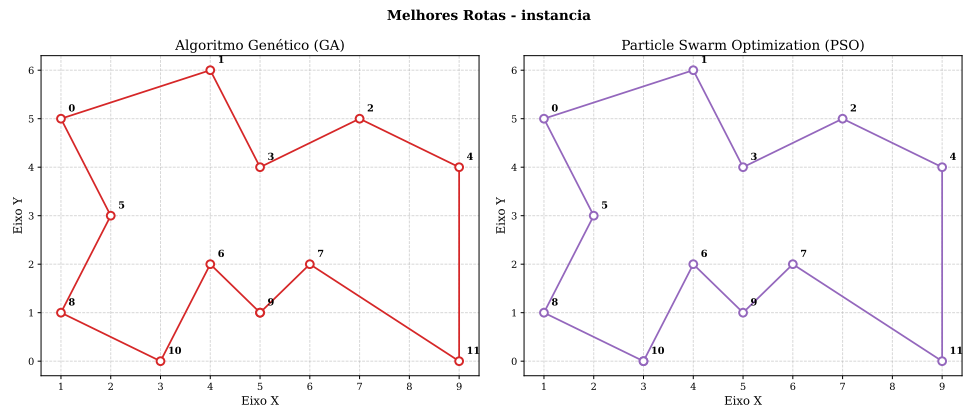


Figura 1. Melhores rotas encontradas para a instância *tsp_12*.

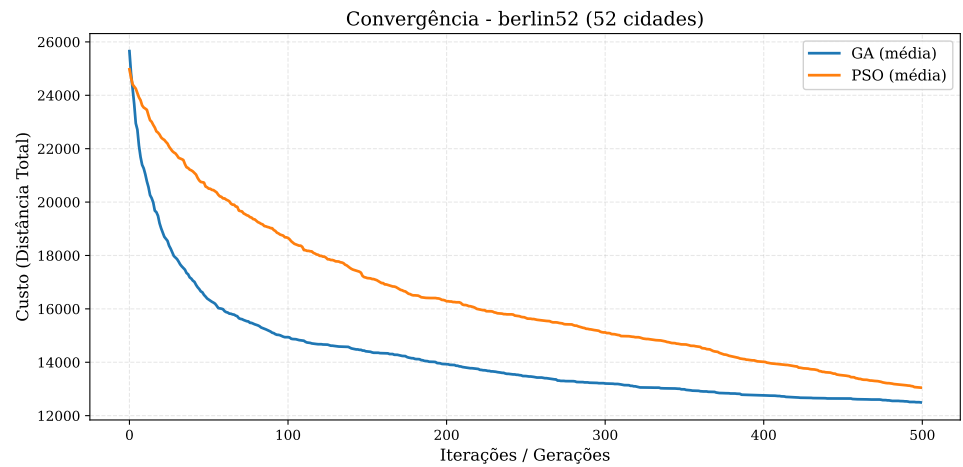


Figura 2. Convergência média do GA e PSO na instância berlin52 (30 execuções).

As Figuras 2 berlin52 e 3 kroA100 revelam um padrão consistente: o GA apresenta uma queda acentuada de custo nas primeiras 50 iterações e se estabiliza relativamente cedo, enquanto o PSO inicia com custo significativamente maior, mas converge progressivamente, reduzindo a diferença para o GA ao longo das iterações. Esse comportamento, ainda que aproxime o PSO do GA, não é suficiente para alcançá-lo no número de iterações testado. Esse padrão pode ser explicado por características intrínsecas das duas meta-heurísticas. O GA, por meio do operador OX combinado com elitismo e seleção por torneio, realiza *saltos macroscópicos* no espaço de soluções já na primeira geração: subsequências promissoras de diferentes indivíduos aleatórios são recombinadas, produzindo filhos que herdam simultaneamente os melhores trechos dos pais. Como consequência, o GA explora rapidamente boas regiões do espaço de busca nos primeiros

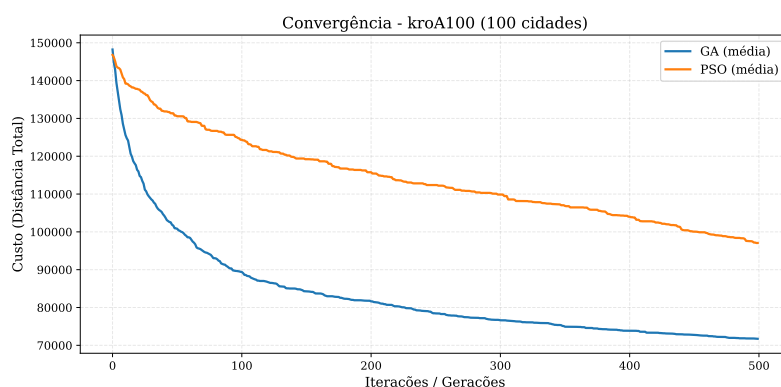


Figura 3. Distribuição dos custos finais para a instância kroA100.

estágios da execução. Após essa fase de melhorias rápidas, o GA tende à estabilização, pois a população converge para regiões similares e o operador OX passa a produzir filhos cada vez mais semelhantes aos pais.

O PSO, por outro lado, parte de uma situação em que as velocidades das partículas são inicializadas vazias e o $pBest$ de cada partícula coincide com sua posição inicial, anulando o componente cognitivo nas primeiras iterações. Adicionalmente, o coeficiente de inércia inicia elevado ($w_{max} = 0,9$), o que faz com que as partículas tendam a manter sua direção corrente — que é nula nesta etapa. Esses fatores combinados resultam em movimentação lenta nas primeiras iterações. À medida que a inércia decai linearmente até ($w_{min} = 0,4$) os componentes cognitivo e social passam a dominar a dinâmica, e as partículas começam a se aproximar das melhores soluções encontradas. Esse mecanismo justifica a aceleração observada no meio da execução e o estreitamento progressivo da diferença em relação ao GA. No entanto, como o operador de movimento do PSO é baseado em *swaps* individuais — operações de granularidade fina, que alteram poucas posições por vez —, mesmo após estabilizar o movimento o PSO converge mais lentamente que o GA, cujo OX manipula subsequências inteiras.

4.4. Discussão

Os resultados experimentais demonstram que o GA apresentou desempenho superior ao PSO em todas as instâncias avaliadas, em todas as métricas consideradas (melhor custo, pior custo, média). O teste de Wilcoxon-Mann-Whitney confirmou que essas diferenças são estatisticamente significativas em todas as instâncias, com p -valor inferior a 0,01.

Comportamento em instâncias pequenas. Na instância `tsp_12`, ambos os algoritmos foram capazes de encontrar a mesma melhor solução (custo de 29,25), o que sugere que essa solução corresponde ao ótimo global. A diferença entre os algoritmos manifesta-se na *consistência*: o GA apresentou desvio padrão de 0,64, enquanto o PSO apresentou 1,13, quase o dobro. Em termos práticos, o GA encontra a solução ótima em uma fração maior das execuções, enquanto o PSO ocasionalmente fica preso em soluções subótimas. Cabe ressaltar, no entanto, que o PSO foi 2,8 vezes mais rápido nessa instância, o que pode ser relevante em aplicações com restrições rigorosas de tempo.

Degradação com o aumento da instância. A diferença relativa entre os algoritmos cresce significativamente com o tamanho da instância. Em `tsp_12` (12 cidades), o PSO apresentou média apenas 1,4% superior à do GA. Em `berlin52` (52 cidades), essa

diferença subiu para 4,4%. Já em *eil101* e *kroA100*, a média do PSO foi respectivamente 28,9% e 35,3% superior à do GA. Esse padrão indica que a abordagem *swap-based* do PSO, embora teoricamente fundamentada, tem dificuldade em escalar: à medida que as rotas ficam maiores, a distância em *swaps* entre uma posição corrente e o *pBest* ou *gBest* cresce, e a sequência de swaps resultante torna-se uma forma menos precisa de aproximação. O operador OX do GA, em contraste, preserva diretamente subsequências dos pais, mantendo informação estrutural mesmo em instâncias maiores.

Distância em relação ao ótimo. A Tabela 5 mostra que, apesar do desempenho superior do GA, *ambos os algoritmos* estão consideravelmente distantes dos ótimos conhecidos das instâncias TSPLIB. O *gap* médio do GA varia de 65% (em *berlin52*) a 237% (em *kroA100*). Isso indica que, embora as implementações estejam corretas e produzam resultados estatisticamente consistentes, os hiperparâmetros adotados não permitem que os algoritmos atinjam soluções próximas do ótimo absoluto em instâncias grandes. Possíveis caminhos para melhoria incluem: aumento substancial do número de iterações; introdução de operadores de busca local (*2-opt*, *3-opt*) intercalados às operações evolutivas; ou utilização de inicialização heurística da população em vez de aleatória.

Tempo de execução. Um padrão interessante surge na análise dos tempos: o PSO é mais rápido em *tsp_12*, mas mais lento nas três instâncias maiores. Em *eil101* e *kroA100*, o PSO é aproximadamente duas vezes mais lento que o GA. Isso se deve à operação de cálculo de diferença entre rotas (sequência de *swaps* mínima), cuja complexidade é $O(n^2)$ pela busca por índice. Trata-se de um ponto que pode ser otimizado em implementações futuras com estruturas auxiliares.

5. Conclusão

Este trabalho apresentou uma comparação sistemática entre o Algoritmo Genético e a Otimização por Enxame de Partículas aplicados ao Problema do Caixeiro Viajante. Ambos os algoritmos foram implementados com representação por caminho, e adotaram-se operadores específicos para o problema combinatório: *Order Crossover* e mutação por posição no GA, e operadores baseados em *swaps* com inércia decrescente e *clamping* de velocidade no PSO. A comparação foi conduzida com rigor estatístico, por meio de 30 execuções por algoritmo em quatro instâncias de tamanhos variados e teste de Wilcoxon-Mann-Whitney para validar a significância das diferenças observadas.

Os resultados indicaram que o GA apresentou desempenho superior ao PSO em todas as instâncias avaliadas, em todas as métricas (melhor custo, pior custo, média e desvio padrão), com diferenças estatisticamente significativas ($p < 0,01$). A diferença relativa entre os algoritmos cresceu com o tamanho da instância, sugerindo que a abordagem *swap-based* do PSO tem maior dificuldade de escalar do que o operador OX do GA. Ambos os algoritmos, no entanto, ficaram consideravelmente distantes dos ótimos conhecidos das instâncias TSPLIB, indicando margem para melhorias.

Como direções para trabalhos futuros, destacam-se: (i) a investigação de topologias alternativas para o PSO, como *Ring* ou *Von Neumann*, que tendem a preservar maior diversidade do enxame e podem mitigar a convergência prematura; (ii) a aplicação de operadores genéticos alternativos, como o *Partially Mapped Crossover* (PMX) e o *Edge Recombination Crossover* (ERX); (iii) a hibridização das duas abordagens, integrando características exploratórias do GA com a estrutura social do PSO; (iv) a incorporação

de busca local (*2-opt*, *3-opt*) como operador adicional, técnica conhecidamente capaz de aproximar significativamente as soluções dos ótimos conhecidos; e (v) a otimização da operação de cálculo de diferença de rotas no PSO, atualmente com complexidade $O(n^2)$.

Referências

- Applegate, D. L., Bixby, R. E., Chvátal, V., and Cook, W. J. (2007). *The traveling salesman problem: a computational study*. Princeton University Press.
- Clerc, M. (2004). Discrete particle swarm optimization, illustrated by the traveling salesman problem. In *New optimization techniques in engineering*, pages 219–239. Springer.
- Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley.
- Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.
- Kennedy, J. and Eberhart, R. (1995). Particle swarm optimization. In *Proceedings of ICNN'95 - International Conference on Neural Networks*, volume 4, pages 1942–1948. IEEE.
- Laporte, G. (1992). The traveling salesman problem: An overview of exact and approximate algorithms. *European Journal of Operational Research*, 59(2):231–247.
- Larrañaga, P., Kuijpers, C. M. H., Murga, R. H., Inza, I., and Dizdarevic, S. (1999). Genetic algorithms for the travelling salesman problem: A review of representations and operators. *Artificial Intelligence Review*, 13(2):129–170.
- Mann, H. B. and Whitney, D. R. (1947). On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics*, 18(1):50–60.
- Reinelt, G. (1991). TspLib—a traveling salesman problem library. *ORSA Journal on Computing*, 3(4):376–384.
- Shi, Y. and Eberhart, R. (1998). A modified particle swarm optimizer. In *1998 IEEE International Conference on Evolutionary Computation Proceedings*, pages 69–73. IEEE.